

Poly Embedder

Davide Gianessi 0001245444

Progetto di "Sistemi di elaborazione accelerata". il codice è disponibile su GitHub all'indirizzo https://github.com/DavideGianessi/poly_embedder

Indice

1. Introduzione	3
1.1. Problema	3
1.2. L'algoritmo	3
1.3. Fase di test	3
2. Implementazione Sequenziale	4
2.1. Operazioni di base	4
2.2. Polinomio annullante	4
2.3. Interpolazione di lagrange	4
2.4. Moltiplicazione con NTT	5
3. Prima implementazione parallela	6
3.1. Polinomio annullante	6
3.2. Interpolazione di Lagrange	6
3.3. Generazione casuale di $R(x)$	7
3.4. Moltiplicazione con NTT	7
3.5. Kernel per la somma finale	8
3.6. Risultati della prima iterazione	8
4. Prima ottimizzazione	9
4.1. risultati	10
5. Seconda ottimizzazione	11
5.1. risultati	11
6. Terza ottimizzazione	13
6.1. risultati	15
7. Conclusioni	16

1. Introduzione

1.1. Problema

Lo scopo del programma è quello di partire da n punti forniti e da un grado desiderato d , e generare un polinomio di grado d che passi per tutti gli n punti.

I $d-n-1$ gradi di libertà rimasti devono essere scelti in modo casuale uniforme, utilizzando un generatore crittograficamente sicuro.

Tutti i valori sono in aritmetica modulare su un campo finito con modulo primo $P=4194304001$, scelto leggermente inferiore a 2^{32} . Il modulo ammette radici dell'unità fino a 2^{24} , necessarie per l'uso della Number Theoretic Transform (NTT).

Di conseguenza, il valore massimo supportato per d è 2^{24} .

1.2. L'algoritmo

L'algoritmo segue i seguenti passi:

- Generazione del polinomio annullatore $V(x)$, tale che $V(x_i)=0$ per ogni x_i $V(x) = (x-x_1)(x-x_2)\dots(x-x_n)$
- Interpolazione di lagrange tra i punti forniti, ottenendo $L(x)$ tale che $L(x_i)=y_i$
- Generazione casuale di un polinomio $R(x)$ di grado $d-n$ tramite un PRNG crittograficamente sicuro
- Moltiplicazione tra $V(x)$ e $R(x)$ tramite NTT $E(x) = V(x) \cdot R(x)$
- Somma finale $F(x) = E(x) + L(x)$

1.3. Fase di test

L'algoritmo dipende dai parametri n e d .

- L'interpolazione ha complessità $O(n^2)$
- La NTT ha complessità $O(d \log d)$

I risultati sperimentali fanno riferimento a un input con: $n = 30000$ $d = 10000000$

	MACCHINA
CPU	AMD Ryzen 7 4800H
GPU	NVIDIA GeForce GTX 1650 Ti Mobile

2. Implementazione Sequenziale

2.1. Operazioni di base

Per evitare il costo delle divisioni nella riduzione modulare si utilizzano:

- predicazione nelle addizioni
- riduzione di Barrett nelle moltiplicazioni

```
pub const P: Fe = 4194304001;
pub const MAGIC: DoubleFe = 4398046510;
pub fn mult(a: Fe, b: Fe) -> Fe {
    let x = a as DoubleFe * b as DoubleFe;
    let q = ((x >> FE_BITS) * MAGIC) >> FE_BITS;
    let r = x - q * (P as DoubleFe);
    let mut r = r as DoubleFe;
    if r >= P as DoubleFe {
        r -= P as DoubleFe;
    }
    if r >= P as DoubleFe {
        r -= P as DoubleFe;
    }
    r as Fe
}
pub fn add(a: Fe, b: Fe) -> Fe {
    let mut x = a as DoubleFe + b as DoubleFe;
    if x >= P as DoubleFe {
        x -= P as DoubleFe;
    }
    x as Fe
}
```

2.2. Polinomio annullante

Otteniamo il polinomio annullante $V(x)$ moltiplicando insieme $(x-x_i)$ per ogni x_i

2.3. Interpolazione di lagrange

$$L(x) = \sum_{i=0}^n y_i \cdot \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} \quad (1)$$

Indichiamo con

$$w_i = \frac{y_i}{\prod_{j=0, j \neq i}^n (x_i - x_j)} \quad (2)$$

e riscriviamo

$$\prod_{j=0, j \neq i}^n (x - x_j) = \frac{V(x)}{x - x_i} \quad (3)$$

calcoliamo quindi prima i pesi w_i , e poi calcoliamo

$$L(x) = \sum_{i=0}^n w_i \cdot \frac{V(x)}{x - x_i} \quad (4)$$

2.4. Moltiplicazione con NTT

La moltiplicazione di polinomi tramite NTT si articola in tre fasi principali:

1. **Trasformata diretta (forward NTT)** entrambi i polinomi di input vengono convertiti dal dominio dei coefficienti al dominio delle radici dell'unità. In pratica, invece di rappresentare il polinomio come somma di potenze di x , lo rappresentiamo come valutazione sulle radici dell'unità.
2. **Moltiplicazione punto a punto** Dopo la trasformata, i due polinomi sono valutati sugli stessi punti. Si moltiplicano quindi punto per punto per ottenere il prodotto dei due polinomi
3. **Trasformata inversa (inverse NTT)** Riporta il polinomio dal dominio delle radici dell'unità al dominio dei coefficienti. La trasformata inversa riutilizza la trasformata diretta, ma utilizza le radici invertite e applica un fattore di scala.

Prima di applicare le “butterfly” della NTT, l'array dei coefficienti viene **riordinato secondo l'ordine dei bit invertiti**.

- Se un indice i ha rappresentazione binaria $b_0 \ b_1 \ b_2 \ \dots \ b_k$, il nuovo indice dell'elemento sarà $b_k \ \dots \ b_2 \ b_1 \ b_0$.
- Questo riordino garantisce che ogni coppia di elementi coinvolta in una butterfly successiva sia già contigua in memoria, evitando conflitti e semplificando l'accesso sequenziale.
- Dopo il bit-reversal, le iterazioni successive applicano le **butterfly $\log(n)$ volte**, raddoppiando la dimensione del blocco ad ogni passo fino a coprire tutto l'array.

—

- Butterfly (iterazioni $\log(n)$)
- Ad ogni iterazione consideriamo blocchi di dimensione crescente ($len = 2, 4, 8 \ \dots \ n$).
- Ogni butterfly combina due metà del blocco usando la radice dell'unità corrispondente, aggiornando i valori in-place.
- La combinazione avviene tramite somme e differenze modulari pesate dalla radice dell'unità, garantendo che alla fine della $\log(n)$ iterazioni l'array rappresenti la trasformata completa.

	SEQUENZIALE
Generating vanishing polynomial	1.465 s
Lagrange interpolation	10.755 s
Generating random polynomial	100.612 ms
NTT multiplication	4.112 s
final sum	7.114 ms

3. Prima implementazione parallela

3.1. Polinomio annullante

Il calcolo della produttoria $\prod_{i=0}^n (x - x_i)$ viene parallelizzato seguendo uno schema ad albero binario. Invece di moltiplicare i monomi uno alla volta, i polinomi vengono moltiplicati a coppie in $\log(n)$ step.

1. Inizializzazione: Si parte da n polinomi di grado 1 della forma $(x - x_i)$. L'array viene completato con polinomi identità fino alla potenza di 2 successiva (n_padded)
2. Riduzione ad albero: ad ogni livello L , si moltiplicano coppie di polinomi di grado 2^L per ottenere polinomi di grado 2^{L+1} .

Ogni thread è responsabile del calcolo di un coefficiente del polinomio risultante.

```
extern "C" __global__ void generate_vanishing(
    const uint32_t* input_array,
    uint32_t* output_array,
    int level,
    int n_padded
) {
    int id = blockIdx.x * blockDim.x + threadIdx.x;
    if (id >= 2*n_padded) return;
    int k = id & ((1<<(level+2))-1);
    int input_1 = id - k;
    int input_2 = input_1 + (1<<(level+1));
    int last_relevant_input = 1<<level;

    uint32_t sum = 0;
    int i_start = max(0, k - last_relevant_input);
    int i_end = min(k, last_relevant_input);

    for (int i = i_start; i <= i_end; i++) {
        int j = k - i;
        sum = fe_add(sum, fe_mul(input_array[input_1 + i],
                                input_array[input_2 + j]));
    }
    output_array[id] = sum;
}
```

3.2. Interpolazione di Lagrange

L'implementazione parallela è stata suddivisa in tre fasi distinte:

Un primo kernel calcola i pesi:

$$w_i = \frac{y}{\prod_{j=0, j \neq i}^n x_i - x_j} \quad (5)$$

poi un secondo kernel calcola il contributo di ogni punto:

$$C_{i(x)} = w_i \cdot \frac{V(x)}{x - x_i} \quad (6)$$

Per calcolare la divisione $\frac{V(x)}{x-x_i}$, Il thread processa i coefficienti di $V(x)$ partendo dal grado più alto e genera i nuovi coefficienti di $C_i(x)$ in tempo reale, mantenendo in un registro solo il resto parziale necessario per il passo successivo della divisione.

Infine un terzo kernel somma tra loro i risultati parziali $C_i(x)$ per ottenere $L_i(x)$.

Tuttavia questo richiede avere un buffer di n elementi per ogni thread, che per n grandi rischia di andare fuori memoria, pertanto ho aggiunto un parametro configurabile `work_per_thread` che definisce di quanti $C_i(x)$ si occupa ciascun thread e accumula su uno stesso buffer. Il valore ottimale di questo parametro è stato determinato sperimentalmente: un valore troppo alto limita il parallelismo, mentre un valore troppo basso può causare il superamento della memoria disponibile o comunque peggiorare la località.

3.3. Generazione casuale di $R(x)$

Per la generazione del polinomio casuale $R(x)$, La libreria standard `CuRand` è stata scartata. Sebbene efficiente per simulazioni statistiche, non garantisce le proprietà di un CSPRNG (Cryptographically Secure Pseudo-Random Number Generator), requisito fondamentale per questo progetto.

Si è scelto di implementare `ChaCha20`, un cifrario molto leggero, per generare i nostri valori partendo da una chiave segreta generata dall'host e passata come argomento al kernel

Ogni thread calcola:

$$R[\text{thread_id}] = \text{ChaCha20}(\text{secret_key}, \text{thread_id}) \quad (7)$$

3.4. Moltiplicazione con NTT

Il primo kernel relativo alla moltiplicazione con NTT è quello che calcola le radici dell'unità (twiddle factors).

Manteniamo come costanti la prima radice dell'unità per ogni livello. In base al valore di d , individuiamo il numero di livelli necessari e preleviamo la prima radice dell'unità relativa al livello massimo. Ogni thread esegue un elevamento a potenza per calcolare il valore di uno di questi twiddles.

se ad esempio $d=24$, abbiamo bisogno fino alle 32-esime radici dell'unità (la prima potenza di 2 $\geq d$) e indicando con w la prima 32-esima radice dell'unità questo kernel ci fornisce il seguente array:

$$1, w, w^2, w^3, w^4, w^5, w^6, w^7, \dots, w^{31} \quad (8)$$

Il secondo kernel svolge la fase di bit-reversal:

```
extern "C" __global__ void bit_reverse(uint32_t* data, uint32_t n, uint32_t
bits) {
    uint32_t i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    uint32_t j = __brev(i) >> (32 - bits);
    if (i < j) {
        uint32_t temp = data[i];
```

```

        data[i] = data[j];
        data[j] = temp;
    }
}

```

Il terzo svolge un singolo step della NTT. Ogni thread legge i due valori della «butterfly» su cui deve operare e il twiddle corrispondente, applicando l'algoritmo in-place.

è importante notare che l'array dei twiddle contiene implicitamente tutti i twiddle dei livelli precedenti. Ad esempio, i twiddle necessari per il livello immediatamente precedente sono rappresentati dagli elementi di indice pari dell'array corrente.

Infine, sono stati implementati i kernel per la moltiplicazione punto a punto e per applicare il fattore di scaling, utilizzato dalla trasformata inversa.

3.5. Kernel per la somma finale

```

extern "C" __global__ void sum_polynomials(
    const uint32_t* poly1, int size1,
    const uint32_t* poly2, int size2,
    uint32_t* result, int result_size
) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= result_size) return;

    uint32_t val = 0;
    if (tid < size1) {
        val = fe_add(val, poly1[tid]);
    }
    if (tid < size2) {
        val = fe_add(val, poly2[tid]);
    }
    result[tid] = val;
}

```

3.6. Risultati della prima iterazione

	SEQUENZIALE	PARALLELA
Moving points to gpu memory		176.77 μ s
Generating vanishing polynomial	1.465 s	8.965 ms
Lagrange interpolation	10.755 s	473.48 ms
Generating random polynomial	100.612 ms	5.62 ms
NTT multiplication	4.112 s	240.547 ms
Final sum	7.114 ms	1.074 ms
Moving result to host		8.2 ms

4. Prima ottimizzazione

Nel kernel dello step NTT, quando il kernel accede ai twiddles di livello inferiore, accede con uno stride, questo porta ad accessi non coalescenti, che peggiorano gravemente le performance, risolviamo questo problema facendo calcolare a `compute_twiddles` un array per ogni livello. Inoltre raccogliamo i primi 11 step in un solo lancio, utilizzando `__syncthreads` invece che una sincronizzazione globale tra uno step e l'altro.

```
extern "C" __global__ void ntt_step(
    uint32_t* poly,
    const uint32_t* twiddles,
    uint32_t n,
    uint32_t log_len,
    uint32_t len,
    uint32_t log_half,
    uint32_t half,
    uint32_t twiddle_offset
) {
    uint32_t tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= (n >> 1)) return;

    uint32_t block_idx = tid >> log_half;
    uint32_t j = tid & (half - 1);
    uint32_t i = block_idx << log_len;
    uint32_t idx1 = i + j;
    uint32_t idx2 = idx1 + half;

    uint32_t w = twiddles[twiddle_offset + j];

    uint32_t u = poly[idx1];
    uint32_t v = fe_mul(poly[idx2], w);

    poly[idx1] = fe_add(u, v);
    poly[idx2] = fe_sub(u, v);
}
```

4.1. risultati

Demangled Name	Duration [ms]
sum_workspaces	1.44
chacha20	7.71
compute_twiddles	3.86
compute_twiddles	3.86
bit_reverse	23.64
ntt_step	0.77
ntt_step	0.77
ntt_step	0.77
ntt_step	0.77
ntt_step	0.77
ntt_step	0.94
ntt_step	0.94
ntt_step	0.94
ntt_step	0.94
ntt_step	1.00
ntt_step	1.11
ntt_step	1.14
ntt_step	1.14
ntt_step	2.95
ntt_step	9.74
ntt_step	9.57
ntt_step	4.24
ntt_step	3.88
ntt_step	4.01
ntt_step	3.87
ntt_step	2.29
ntt_step	1.48
ntt_step	1.13
ntt_step	0.95

Demangled Name	Duration [ms]
sum_workspaces	1.39
chacha20	7.71
compute_twiddles	7.27
compute_twiddles	7.27
bit_reverse	23.55
bit_reverse	23.51
ntt_block	3.08
ntt_step	0.77
ntt_step	0.77
ntt_step	0.77
ntt_step	0.77
ntt_step	0.77
ntt_step	0.78
ntt_step	0.95
ntt_step	0.95
ntt_step	0.95
ntt_step	0.95
ntt_step	0.95
ntt_step	0.95
ntt_step	0.95
ntt_step	0.95
ntt_step	0.95

	SEQUENZIALE	BASLINE	OTTIMIZZATA
Moving points to gpu memory		176.77 μ s	135.42 μ s
Generating vanishing polynomial	1.465 s	8.965 ms	8.95 ms
Lagrange interpolation	10.755 s	473.48 ms	480 ms
Generating random polynomial	100.612 ms	5.62 ms	5.61 ms
NTT multiplication	4.112 s	240.547 ms	124.57 ms
Final sum	7.114 ms	1.074 ms	1.078 ms
Moving result to host		8.2 ms	7.967 ms

5. Seconda ottimizzazione

La fase di bit-reverse presenta una sfida per le performance: se effettuiamo una lettura coalescente, i valori devono essere scritti in posizioni di memoria globali molto distanti tra loro, rompendo la coalescenza in scrittura. Per superare questo limite, ho sfruttato la **shared memory** e l'accesso tramite **banks**.

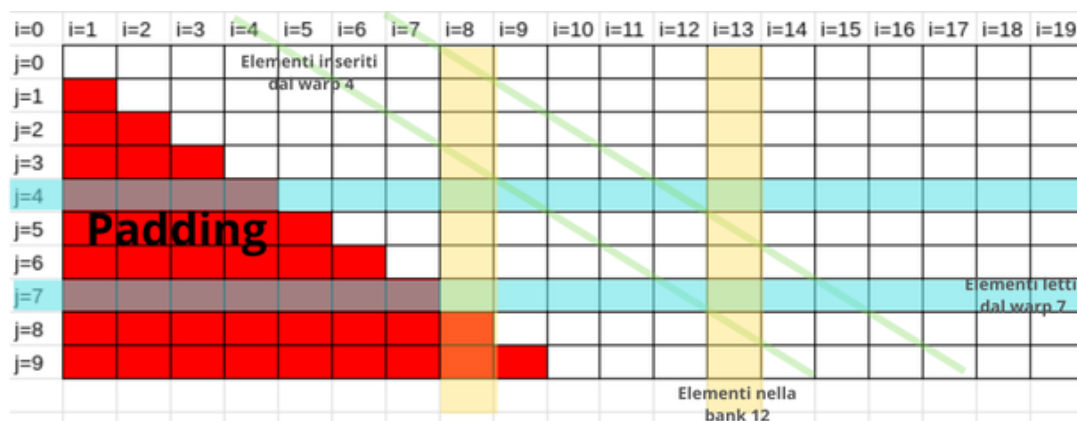
Utilizziamo dei blocchi da 1024 thread (32 warp) e dividiamo l'esecuzione in due fasi:

1. **fase di lettura**: ogni warp esegue una lettura coalescente dalla memoria globale e scrive i valori in shared memory

facciamo una sincronizzazione locale tra le due fasi con `__syncthreads()`

2. **fase di scrittura** ogni warp legge dalla shared memory in modo che ogni thread ottenga un valore originariamente scritto da un warp diverso. Questo permette di riordinare i dati internamente al blocco per eseguire una scrittura finale in memoria globale che sia nuovamente coalescente.

Per evitare **bank conflicts**, che si verificherebbero se più thread accedessero simultaneamente allo stesso banco di memoria, ho introdotto degli offset (padding) ad ogni riga del buffer temporaneo in shared memory.



La logica di indirizzamento divide i bit dell'indice in tre parti: i primi 5 bit formano l'indice di colonna i , gli ultimi 5 bit l'indice di riga j , e i bit intermedi identificano il blocco.

- Nella fase di lettura, ogni warp fissa i ed effettua una lettura coalescente, scrivendo i risultati nella colonna i , diagonale a causa del padding.
- Nella fase di scrittura, ogni warp fissa j , legge i valori della riga corrispondente, e li scrive agli indirizzi risultanti dal bit-reversal

5.1. risultati

	SEQUENZIALE	BASLINE	OTTIMIZZATA
Moving points to gpu memory		176.77 μ s	162.73 μ s
Generating vanishing polynomial	1.465 s	8.93 ms	8.95 ms
Lagrange interpolation	10.755 s	473.48 ms	475 ms
Generating random polynomial	100.612 ms	5.62 ms	5.59 ms

	SEQUENZIALE	BASLINE	OTTIMIZZATA
NTT multiplication	4.112 s	240.547 ms	64.79 ms
Final sum	7.114 ms	1.074 ms	1.076 ms
Moving result to host		8.2 ms	8.17 ms

6. Terza ottimizzazione

L'ultima ottimizzazione riguarda il calcolo dei contributi di lagrange. Il problema principale risiede nella necessità sommare i contributi di tutti i punti; tuttavia, la catena di dipendenza data dalla divisione polinomiale costringe ogni thread a calcolare tutti i contributi relativi ad uno stesso punto, impedendoci di partizionarlo per indice del risultato, da qui la necessità di creare più buffer e poi sommarli successivamente.

L'uso di atomics è stato scartato: l'aritmetica modulare richiederebbe atomiche a 64bit (più lente), e soprattutto tutti i thread calcolano i coefficienti nello stesso ordine, portando ad una contention insostenibile.

La soluzione adottata sfrutta la collaborazione a livello di warp tramite le warp-level primitives. I thread di un warp si scambiano i valori dei registri per accumulare i contributi prima della scrittura, riducendo drasticamente il numero di buffer necessari (da uno per thread a uno per warp).

L'algoritmo è strutturato come una pipeline in tre fasi:

1. **Warm-up:** I thread eseguono iterazioni a vuoto sfalsate per distribuirsi su coefficienti diversi della divisione
2. **Steady State (A regime):** si crea un «nastro trasportatore». Il thread i esimo riceve dal thread $(i-1)$ esimo, che ha appena processato il grado che deve processare adesso l' i esimo, sia il coefficiente del polinomio annullatore di quel grado, che l'accumulatore a cui aggiungere il contributo, all'iterazione successiva lo passerà al thread $(i+1)$ esimo.
 - Ogni 32 iterazioni viene fatta una lettura coalescente che mette i prossimi coefficienti da processare nella variabile `next_input`, e una scrittura coalescente che scrive gli accumulatori nella variabile `ready_acc` che hanno completato il giro di tutti i warp.
 - Il thread 0 (di ogni warp), in quanto thread più avanti nell'esecuzione svolge la funzione di gateway: inizializza i nuovi accumulatori a zero, introduce nuovi valori da `next_input` e sposta gli accumulatori che hanno finito il giro in `ready_acc`
3. **Cool-down:** Svuota la pipeline per processare gli ultimi 32 coefficienti e completare le ultime scritture.

Inizializzazione e fase di warmup:

```
const int lane = threadIdx.x & 31;
const int warp_id = (blockIdx.x * (blockDim.x / 32)) + (threadIdx.x / 32);
if (warp_id >= num_workspaces) return;
uint32_t* my_out = &workspaces[warp_id * n_padded];

const int point_idx = warp_id * 32 + lane;

uint32_t root = (point_idx < n) ? points_x[point_idx] : 0;
uint32_t weight = (point_idx < n) ? weights[point_idx] : 0;

uint32_t current = (n < n_padded) ? 0: vanishing_poly[n_padded];

uint32_t acc = 0;
uint32_t ready_acc = 0;
uint32_t vanish_coeff = 0;
```

```

int first_coeff_idx = n_padded - 32 + ((lane+31)&31);
if (first_coeff_idx <= n) {
    vanish_coeff = vanishing_poly[first_coeff_idx];
}

for (int i = 0; i < 32; i++) {
    if (lane <= i) {
        acc = fe_add(acc, fe_mul(current, weight));
        current = fe_add(vanish_coeff, fe_mul(current, root));
    }
    acc = __shfl_sync(0xffffffff, acc, (lane + 31) & 31);
    vanish_coeff = __shfl_sync(0xffffffff, vanish_coeff, (lane + 31)
& 31);
}

```

fase a regime:

```

for (int r = 1; 32 * r < n_padded; r++) {
    uint32_t next_vanish_coeff = vanishing_poly[n_padded - 32*(r+1) +
((lane+31)&31)];

    for (int i = 0; i < 32; i++) {
        if (lane == 0) {
            ready_acc = acc;
            acc = 0;
            vanish_coeff = next_vanish_coeff;
        }

        acc = fe_add(acc, fe_mul(current, weight));
        current = fe_add(vanish_coeff, fe_mul(current, root));

        acc = __shfl_sync(0xffffffff, acc, (lane + 31) & 31);
        ready_acc = __shfl_sync(0xffffffff, ready_acc, (lane + 31) & 31);
        vanish_coeff = __shfl_sync(0xffffffff, vanish_coeff, (lane + 31) & 31);
        next_vanish_coeff = __shfl_sync(0xffffffff, next_vanish_coeff, (lane
+ 31) & 31);
    }

    int write_idx = n_padded - (32 * (r - 1) + ((32 - lane) & 31)) - 1;
    if (write_idx >= 0 && write_idx < n_padded) {
        my_out[write_idx] = ready_acc;
    }
}

```

fase di cool down:

```

int last_r = n_padded / 32;
for (int i = 0; i < 32; i++) {
    if (lane == 0) {
        ready_acc = acc;
    }

    if (lane > i) {
        acc = fe_add(acc, fe_mul(current, weight));
        current = fe_add(vanish_coeff, fe_mul(current, root));
    }
}

```

```

}

acc = __shfl_sync(0xffffffff, acc, (lane + 31) & 31);
ready_acc = __shfl_sync(0xffffffff, ready_acc, (lane + 31) & 31);
vanish_coeff = __shfl_sync(0xffffffff, vanish_coeff, (lane + 31) & 31);
}

int final_write_idx = n_padded - (32 * (last_r - 1) + ((32 - lane) & 31)) - 1;
if (final_write_idx >= 0 && final_write_idx < n_padded) {
    my_out[final_write_idx] = ready_acc;
}

```

6.1. risultati

	SEQUENZIALE	BASLINE	OTTIMIZZATA
Moving points to gpu memory		176.77 μ s	137.79 μ s
Generating vanishing polynomial	1.465 s	8.93 ms	9.08 ms
Lagrange interpolation	10.755 s	473.48 ms	48.72 ms
Generating random polynomial	100.612 ms	5.62 ms	7.99 ms
NTT multiplication	4.112 s	240.547 ms	70.79 ms
Final sum	7.114 ms	1.074 ms	965 μ s
Moving result to host		8.2 ms	7.93 ms

7. Conclusioni

L'implementazione parallela ottimizzata su GPU dimostra un'accelerazione significativa dei moduli computazionalmente più onerosi rispetto alla baseline parallela e, in modo ancora più marcato, rispetto alla versione sequenziale:

1. NTT: miglioramento di $3.4\times$ (da 240.55 ms a 70.79 ms) grazie a:
 - Riorganizzazione dei twiddle factors.
 - Fusione dei primi 11 step.
 - Ottimizzazione del bit-reverse tramite l'uso della shared memory.
2. Interpolazione di Lagrange: miglioramento di $9.7\times$ (da 473.48 ms a 48.72 ms) grazie a:
 - Uso di warp-level primitives (`__shfl_sync`)
 - Riduzione dei buffer e degli accessi in memoria

Le performance complessive evidenziano un'accelerazione di due ordini di grandezza rispetto alla versione sequenziale:

- Tempo totale sequenziale: 16.44 secondi
- Tempo totale parallelo ottimizzato: 0.14 secondi